

PHASE 8

Appendices

Appendices A–D: API endpoints, environment variables, project structure and sample code

FITBUDDY • AI FITNESS PLAN GENERATOR • PROJECT REPORT

Appendix A: API Endpoints

Method	Endpoint	Purpose
GET	/	Application entry
POST	/generate-workout	Generate workout and nutrition
POST	/submit-feedback	Generate revised workout
GET	/view-all-users	Admin user information
GET	/docs	FastAPI Swagger UI

Appendix B: Environment Variables

Variable	Purpose
GOOGLE_API_KEY	Secret key authenticating server requests to the Gemini API. Must never be committed to version control or shown to users.
GEMINI_WORKOUT_MODEL	Identifier of the Gemini model used for workout generation and revision. Configurable because model names and availability change.
GEMINI_NUTRITION_MODEL	Identifier of the Gemini Flash model used for nutrition and recovery tips.
DATABASE_URL	Connection string for the database, for example a SQLite file path. Changing it, with the matching driver, is the first step in migrating to another database.

Example .env.example content, with placeholder values only:

```
GOOGLE_API_KEY=your_api_key_here
GEMINI_WORKOUT_MODEL=your_workout_model_id
GEMINI_NUTRITION_MODEL=your_flash_model_id
DATABASE_URL=sqlite:///./fitbuddy.db
```

Appendix C: Project Structure

```
fitbuddy/
├── app/
│   ├── main.py
│   ├── database.py
│   ├── schemas.py
│   ├── routes.py
│   ├── gemini_generator.py
│   ├── gemini_flash_generator.py
│   └── updated_plan.py
├── templates/
│   ├── index.html
│   ├── result.html
│   └── all_users.html
├── static/
│   └── style.css
├── .env
└── requirements.txt
```

Responsibilities of the principal files are described in Chapter 7.

Appendix D: Sample Code

This appendix presents a simplified implementation of the FitBuddy system. The code covers the core features described in the project report and is formatted for academic reference and demonstration.

D.1 Project Structure

Project Structure as same as Appendix C Without any changes, Mentioned above.

D.2 Database Module — database.py

Purpose: Creates the SQLite database and defines the SQLAlchemy database models.

```

from sqlalchemy import create_engine, Column, Integer, String, Text
from sqlalchemy.orm import declarative_base, sessionmaker

DATABASE_URL = "sqlite:///./fitbuddy.db"

engine = create_engine(
    DATABASE_URL,
    connect_args={"check_same_thread": False}
)

SessionLocal = sessionmaker(bind=engine)
Base = declarative_base()

class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True)
    user_id = Column(String, unique=True)
    name = Column(String)
    age = Column(Integer)
    weight = Column(Integer)
    goal = Column(String)
    intensity = Column(String)

class WorkoutPlan(Base):
    __tablename__ = "workout_plans"

    id = Column(Integer, primary_key=True)
    user_id = Column(String)
    original_plan = Column(Text)
    nutrition_tip = Column(Text)

class Feedback(Base):
    __tablename__ = "feedback"

    id = Column(Integer, primary_key=True)
    user_id = Column(String)
    feedback = Column(Text)
    revised_plan = Column(Text)

Base.metadata.create_all(bind=engine)

```

D.3 Input Schema Module — schemas.py

Purpose: Defines Pydantic models for validating user information and feedback.

```

from pydantic import BaseModel

class UserInput(BaseModel):
    user_id: str
    name: str
    age: int
    weight: float
    goal: str
    intensity: str

class FeedbackRequest(BaseModel):
    user_id: str
    feedback: str

```

D.4 Gemini Workout Generator — gemini_generator.py

Purpose: Uses Gemini to generate a personalized seven-day workout plan.

```

import os
from google import genai

client = genai.Client(
    api_key=os.getenv("GOOGLE_API_KEY")
)

def generate_workout(user):
    prompt = f"""
    Create a simple 7-day workout plan.

    Name: {user.name}
    Age: {user.age}
    Weight: {user.weight}
    Goal: {user.goal}
    Intensity: {user.intensity}

    Give:
    Day 1 to Day 7
    Exercise names
    Sets
    Repetitions
    Rest time
    Keep the plan safe and simple.
    """

    response = client.models.generate_content(
        model=os.getenv("GEMINI_WORKOUT_MODEL"),
        contents=prompt
    )

    return response.text

```

D.5 Gemini Nutrition Generator — gemini_flash_generator.py

Purpose: Generates a short nutrition and recovery recommendation.

```

import os
from google import genai

client = genai.Client(
    api_key=os.getenv("GOOGLE_API_KEY")
)

def generate_nutrition_tip(user):
    prompt = f"""
    Give a short nutrition and recovery tip.

    Age: {user.age}
    Weight: {user.weight}
    Goal: {user.goal}
    Intensity: {user.intensity}

    Include hydration, healthy food and recovery advice.
    Keep it short.
    """

    response = client.models.generate_content(
        model=os.getenv("GEMINI_NUTRITION_MODEL"),
        contents=prompt
    )

    return response.text

```

D.6 Feedback-Based Plan Revision — updated_plan.py

Purpose: Uses the original workout plan and user feedback to generate a revised plan.

```
import os
from google import genai

client = genai.Client(
    api_key=os.getenv("GOOGLE_API_KEY")
)

def update_workout_plan(user, old_plan, feedback):

    prompt = f"""
    Revise the following workout plan.

    User:
    Age: {user.age}
    Weight: {user.weight}
    Goal: {user.goal}
    Intensity: {user.intensity}

    Original Plan:
    {old_plan}

    User Feedback:
    {feedback}

    Create an improved 7-day workout plan based on the feedback.
    """

    response = client.models.generate_content(
        model=os.getenv("GEMINI_WORKOUT_MODEL"),
        contents=prompt
    )

    return response.text
```

D.7 FastAPI Routes — routes.py

Purpose: Connects the web interface, database, Gemini modules, and feedback workflow.

```

from fastapi import APIRouter, Request, Form
from fastapi.responses import HTMLResponse
from fastapi.templating import Jinja2Templates

from .database import SessionLocal, User, WorkoutPlan, Feedback
from .gemini_generator import generate_workout
from .gemini_flash_generator import generate_nutrition_tip
from .updated_plan import update_workout_plan

router = APIRouter()
templates = Jinja2Templates(directory="templates")

@router.get("/", response_class=HTMLResponse)
def home(request: Request):
    return templates.TemplateResponse(
        "index.html",
        {"request": request}
    )

@router.post("/generate-workout", response_class=HTMLResponse)
def generate(
    request: Request,
    user_id: str = Form(...),
    name: str = Form(...),
    age: int = Form(...),
    weight: float = Form(...),
    goal: str = Form(...),
    intensity: str = Form(...)
):
    db = SessionLocal()

    user = User(
        user_id=user_id,
        name=name,
        age=age,
        weight=weight,
        goal=goal,
        intensity=intensity
    )

    db.add(user)
    db.commit()

    workout = generate_workout(user)
    nutrition = generate_nutrition_tip(user)

```

```

    plan = WorkoutPlan(
        user_id=user_id,
        original_plan=workout,
        nutrition_tip=nutrition
    )

    db.add(plan)
    db.commit()

    return templates.TemplateResponse(
        "result.html",
        {
            "request": request,
            "user": user,
            "workout": workout,
            "nutrition": nutrition
        }
    )

@router.post("/submit-feedback", response_class=HTMLResponse)
def submit_feedback(
    request: Request,
    user_id: str = Form(...),
    feedback: str = Form(...)
):
    db = SessionLocal()

    user = db.query(User).filter(
        User.user_id == user_id
    ).first()

    plan = db.query(WorkoutPlan).filter(
        WorkoutPlan.user_id == user_id
    ).first()

    revised = update_workout_plan(
        user,
        plan.original_plan,
        feedback
    )

    record = Feedback(
        user_id=user_id,
        feedback=feedback,
        revised_plan=revised
    )

    db.add(record)
    db.commit()

    return templates.TemplateResponse(
        "result.html",
        {
            "request": request,
            "user": user,
            "workout": plan.original_plan,
            "nutrition": plan.nutrition_tip,
            "revised": revised
        }
    )

@router.get("/view-all-users", response_class=HTMLResponse)
def view_users(request: Request):
    db = SessionLocal()
    users = db.query(User).all()

    return templates.TemplateResponse(
        "all_users.html",
        {
            "request": request,
            "users": users
        }
    )

```

D.8 Application Entry Point — main.py

Purpose: Creates the FastAPI application and registers the routes.

```
from fastapi import FastAPI
from .routes import router

app = FastAPI(
    title="FitBuddy - AI Fitness Plan Generator"
)

app.include_router(router)
```

D.9 Home Page — templates/index.html

Purpose: Collects the user's fitness information.

```
<!DOCTYPE html>
<html>
<head>
  <title>FitBuddy</title>
</head>

<body>

<h1>FitBuddy</h1>
<h2>AI Fitness Plan Generator</h2>

<form action="/generate-workout" method="post">

  <label>User ID</label>
  <input type="text" name="user_id" required>

  <br><br>

  <label>Name</label>
  <input type="text" name="name" required>

  <br><br>

  <label>Age</label>
  <input type="number" name="age" required>

  <br><br>

  <label>Weight</label>
  <input type="number" name="weight" required>

  <br><br>

  <label>Fitness Goal</label>
  <select name="goal">
    <option>Weight Loss</option>
    <option>Muscle Gain</option>
    <option>Strength</option>
    <option>Endurance</option>
    <option>General Fitness</option>
  </select>

  <br><br>

  <label>Intensity</label>
  <select name="intensity">
    <option>Beginner</option>
    <option>Moderate</option>
    <option>Advanced</option>
  </select>

  <br><br>

  <button type="submit">Generate Workout</button>

</form>

</body>
</html>
```

D.10 Result Page — templates/result.html

Purpose: Displays the generated workout plan and nutrition recommendation and accepts feedback for creating a revised plan.

```
<!DOCTYPE html>
<html>
<head>
  <title>FitBuddy Result</title>
</head>

<body>

<h1>Your Fitness Plan</h1>

<h2>Workout Plan</h2>
<pre>{{ workout }}</pre>

<h2>Nutrition & Recovery Tip</h2>
<p>{{ nutrition }}</p>

{% if revised %}
<hr>
<h2>Updated Workout Plan</h2>
<pre>{{ revised }}</pre>
{% endif %}

<hr>

<h2>Give Feedback</h2>

<form action="/submit-feedback" method="post">

  <input type="hidden"
    name="user_id"
    value="{{ user.user_id }}">

  <textarea
    name="feedback"
    rows="5"
    cols="50"
    placeholder="Tell us how the plan can be improved..."
    required>
  </textarea>

  <br><br>

  <button type="submit">
    Generate Revised Plan
  </button>

</form>
</body>
</html>
```

D.11 Admin Page — templates/all_users.html

Purpose: Displays stored user information for administrative viewing.

```

<!DOCTYPE html>
<html>
<head>
  <title>FitBuddy Admin</title>
</head>

<body>

<h1>FitBuddy Admin Dashboard</h1>

<table border="1">

<tr>
  <th>ID</th>
  <th>User ID</th>
  <th>Name</th>
  <th>Age</th>
  <th>Weight</th>
  <th>Goal</th>
  <th>Intensity</th>
</tr>

{% for user in users %}

<tr>
  <td>{{ user.id }}</td>
  <td>{{ user.user_id }}</td>
  <td>{{ user.name }}</td>
  <td>{{ user.age }}</td>
  <td>{{ user.weight }}</td>
  <td>{{ user.goal }}</td>
  <td>{{ user.intensity }}</td>
</tr>

{% endfor %}

</table>

</body>
</html>

```

D.12 Environment Configuration — .env

Purpose: Stores the Gemini API configuration outside the source code.

```

GOOGLE_API_KEY=your_api_key_here
GEMINI_WORKOUT_MODEL=your_workout_model
GEMINI_NUTRITION_MODEL=your_nutrition_model

```

D.13 Dependencies — requirements.txt

```

fastapi
uvicorn
jinja2
python-multipart
sqlalchemy
pydantic
python-dotenv
google-genai

```

D.14 Running The Application

Step 1 – Install dependencies:

```
pip install -r requirements.txt
```

Step 2 – Start the FastAPI server:

```
uvicorn app.main:app --reload
```

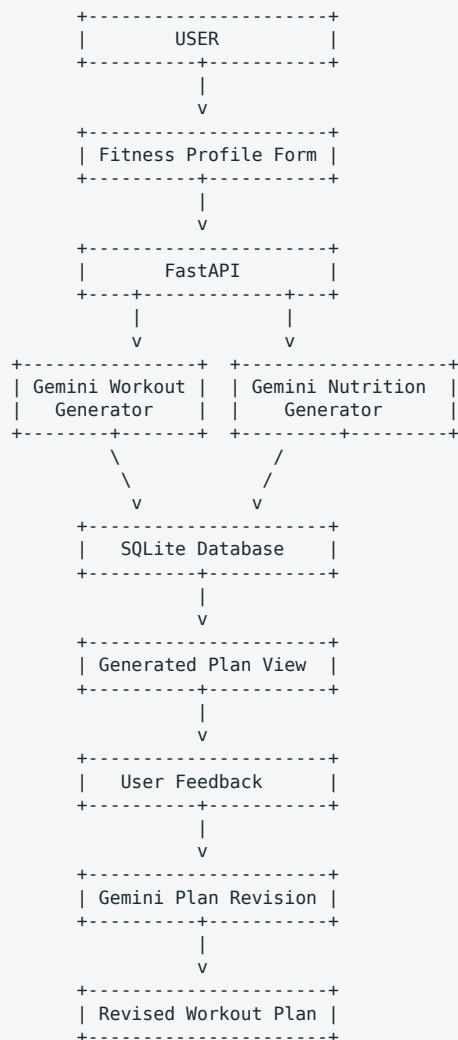
Step 3 – Open the application:

```
http://127.0.0.1:8000
```

Step 4 – Open the FastAPI documentation:

```
http://127.0.0.1:8000/docs
```

D.15 System Implementation Flow



D.16 Implementation Summary

The simplified implementation demonstrates the following workflow:

1. The user enters basic fitness information.
2. FastAPI receives and processes the submitted information.

3. Gemini generates a personalized seven-day workout plan.
4. A nutrition and recovery recommendation is generated.
5. The generated information is stored in SQLite.
6. The original plan is displayed to the user.
7. The user can submit feedback.
8. Gemini uses the original plan and feedback to generate a revised plan.
9. The revised plan is stored separately so that the original plan is retained.
10. The administrative page can display stored user information.

NOTE:

This implementation is intended for academic demonstration. A production implementation should additionally provide stronger authentication and authorization, robust exception handling, structured AI-output validation, database relationships and migrations, rate limiting, secure session handling, and comprehensive automated testing.